

13. System Design

13.1 Use Case Diagram

The Use Case Diagram below represents the dynamic model of the Medical Appointment & Prescription System. It describes the interaction between the system users (Actors) and the functions (Use Cases) provided by the application.

- **System Boundary** – the dashed rectangle, which represents the software application.
- **Actors** – the external entities, which interact with the system. In the Medical Appointment & Prescription System, the Actors are Admin, Staff, and Doctor.
- **Use Cases** – the functions offered by the system, which are depicted as ovals. They are placed within the System Boundary.

i. Use Case Components & Symbols

Element Name	Visual Representation / Symbol	Description
Actor		Means an entity that interacts with the system, such as users, external hardware, or other systems.
Use Case		Means a discrete function or activity that enables realization of a particular purpose or action of an actor.
System Boundary		Defines the scope of the system; everything inside the box belongs to the application, while the actors are outside
Associate Line		Indicates participation or communication between an actor and a specific use case.

ii. Detailed Actor Descriptions

- **Admin:**
 - ◆ The system administrator who configures the system, manages staff and doctor accounts, and maintains directory information.
- **Staff (Receptionist / Front Desk):**
 - ◆ Front desk staff members who perform day-to-day administrative tasks, such as check-in of patients and organization of the waiting list.
- **Doctor (Medical Practitioner):**
 - ◆ Doctors who examine patients, study their cases, prescribe medication in electronic format, and prepare reports.

iii. Comprehensive Use Case Descriptions

- **Use Case 1: Manage Doctor**

- ◆ **Actor:** Admin
- ◆ **Description:** This use case enables the admin to add, update or delete any information relating to the doctors, including their specialization, consultation fee, and working hours in the database.
- ◆ **Precondition:** Admin has accessed the master control panel.
- ◆ **Postcondition:** The database contains updated information on doctors.

- **Use Case 2: Manage Staff**

- ◆ **Actor:** Admin
- ◆ **Description:** This use case allows the system administrator to create, update, or revoke receptionist and employee login credentials and role permissions.
- ◆ **Precondition:** The admin is authenticated into administrative control panel.
- ◆ **Postcondition:** Staff user accounts and role authorizations are updated in the Users table.

- **Use Case 3: Manage Patient**

- ◆ **Actor:** Staff
- ◆ **Description:** With this use case, staff can add new patients, assigning them unique patient IDs, and update their information.
- ◆ **Precondition:** The staff has accessed the receptionist dashboard.
- ◆ **Postcondition:** Patient's information has been added to the database.

- **Use Case 4: Book Appointment**

- ◆ **Actors:** Staff, Doctor
- ◆ **Description:** This use case enables staff to book appointments for patients by selecting the suitable doctor, date, and time for consultation while avoiding any schedule clashes.
- ◆ **Precondition:** The patient's profile has been created, and the doctor has updated his/her working hours.
- ◆ **Postcondition:** Appointment has been created with a status of Pending or Confirmed.

- **Use Case 5: Create Prescription (Note: Create Description / Prescription)**

- ◆ **Actor:** Doctor
- ◆ **Description:** Doctors can create electronic prescriptions for patients, specifying the drugs, dosage, intervals between doses, treatment length, and additional notes.
- ◆ **Precondition:** Doctor has accessed the medical records of the patient, and they have an appointment.
- ◆ **Postcondition:** Information on the prescribed medications has been saved in the database.

- **Use Case 6: Patient History**

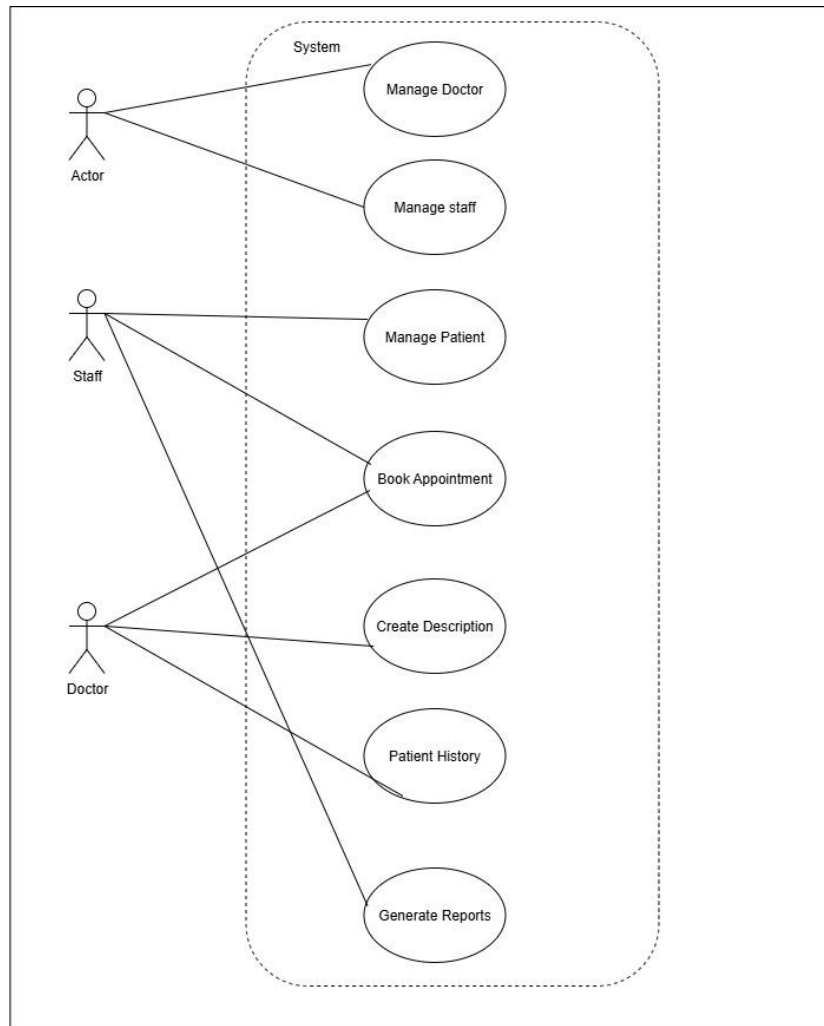
- ◆ **Actor:** Doctor
- ◆ **Description:** This use case enables doctors to view the medical history of their patients, including the records of any previous prescriptions and treatments.
- ◆ **Precondition:** Doctor has accessed the medical records of the patient.
- ◆ **Postcondition:** Medical history appears on the dashboard.

- **Use Case 7: Generate Reports**

- ◆ **Actor:** Staff, Doctor

- ◆ **Description:** The user can generate reports containing patient information, appointment, and prescription details in PDF format for printing.
- ◆ **Precondition:** Patient's information, appointment, and prescription records have been updated in the database.
- ◆ **Postcondition:** Report has been saved as a PDF file.

iv. Diagram



13.2 Class Diagram

A Class Diagram is a unified modeling language (UML) class diagram that describes the static structure of the system. It displays the system's classes, their attributes, and the relationships between the classes. This is the basis for object-oriented design and implementation.

i. Class Diagram Components & Symbols

Élément Name	Visual Representation / Symbol	Description
Class		Draw the UML diagram of the system's classes (User, Patient, Doctor, Appointment, Prescription) in the section below. You should divide the classes into subdivisions with the class's name on top and the attributes below.
Associations		Show the relationships and navigation hints between the

		classes by marking how the objects can be reached from a particular instance (e.g., User down to Patient and Doctor or Appointment to Prescription).
--	--	--

ii. Structure of Class Boxes

- In the class diagram, each class is represented by a box, divided into two parts:
- The top part contains the class name (e.g., User, Patient)
- The bottom part contains the set of attributes (data fields) that describe this class; in other words, the information that characterizes it.

iii. Detailed Description of the System Classes

● User Class

- ◆ The user class is the primary authentication entity that controls system access and security clearance for all software end-users.
- ◆ The attributes include a unique identification number, login name, and access role. The class acts as the basic foundation by defining the authorization level before assigning roles to different users in the system.

● Patient Class

- ◆ The patient class defines the individuals who come to the hospital for consultation, treatment, or medication.
- ◆ The attributes include the patient's personal name, age, and contact information. It acts as a primary record holder of all patients' history of visits and appointment schedules.

● Doctor Class

- ◆ The doctor class is the professional category of staff involved in examining, attending, and dispensing treatment to patients.
- ◆ The attributes comprise the doctor's name and area of specialization or department. The class serves as an intermediate level between the staff and the patient's management system.

● Appointment Class

- ◆ The appointment class deals with scheduling and management of appointments for visiting doctors at allocated time slots.
- ◆ The attributes comprise the date and time of the scheduled visit and the status of the appointment. It serves as a reminder mechanism and prevents appointment overlaps by managing patient arrivals into the hospital.

● Prescription Class

- ◆ The prescriptions class stores information relating to the treatment instructions and medication notes provided after clinical examinations.
- ◆ The attributes comprise the drugs and doses administered to patients and any additional notes provided by the physician. It provides a historical record of all medication administered to a specific patient.

iv. Class Associations and Inter-Object Relationships

- **User to Patient & Doctor (Authentication Mapping)**

- ◆ **Authentication boundary:** This association acts as the core security layer where the abstracted User class controls system access, authentication credentials, and permission levels.
- ◆ **Profile mapping:** This connects downwards to the Patient and Doctor classes to establish account credentials in accordance with their respective user profiles.
- ◆ **Data integrity:** Ensures each session is tied to an existing medical or administrative entity within the application's ecosystem.

- **Patient & Doctor to Appointment (Consultation Association)**

- ◆ **Coordination layer:** Binds the Patient and Doctor classes to the Appointment class to facilitate consultation booking procedures.
- ◆ **Resource allocation:** Allocates a particular patient searching for healthcare services to a doctor available for consultation on a given day and time.
- ◆ **Conflict prevention:** Operates as the working bridge that monitors schedules to avoid overbooking or conflicting appointments.

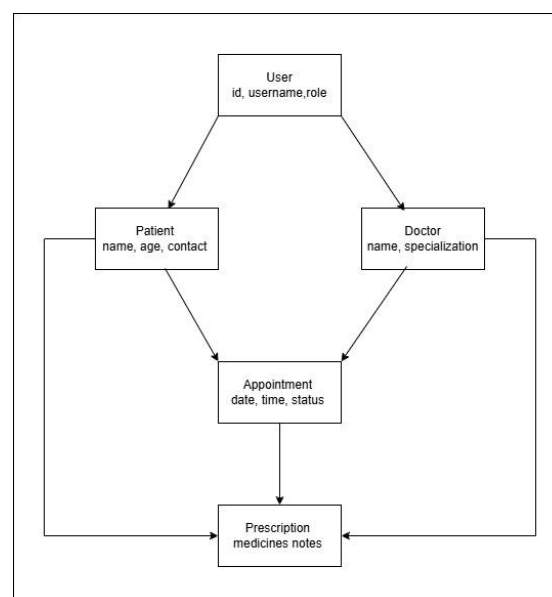
- **Appointment to Prescription (Operational Dependency)**

- ◆ **Sequential transition:** This association acts as a direct operational trigger that initiates the Prescription class when an Appointment is successfully created.
- ◆ **Clinical dependency:** Ties the medical records directly back to the corresponding appointment and timestamp when the treatment plan was created.
- ◆ **System traceability:** This allows the software to trace back every medication issued to a patient in line with their appointment and the verified log visit

- **Patient & Doctor to Prescription (Historical & Issuance Link)**

- ◆ **Record issuance:** This links the Doctor class directly to the Prescription class to enable medical practitioners to record treatment notes.
- ◆ **Historical retention:** Binds the Patient class directly to the Prescription class to store a chronological treatment history of medications dispensed.
- ◆ **Continuity of care:** Enables a doctor to quickly review a patient's treatment plan and continue treatment safely and effectively.

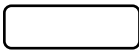
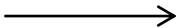
v. Diagram



13.3 Activity Diagram

An Activity Diagram is a behavioral UML (Unified Modeling Language) diagram, which depicts the flow of control from one activity to another. It is basically an enhanced version of a flowchart and portrays the procedural logic or the business workflow of the system.

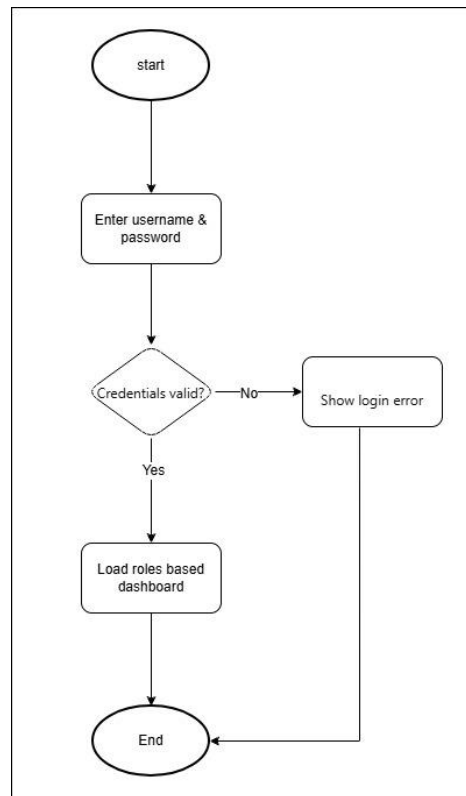
i. Activity Diagram Components & Symbols

Élément Name	Visual Representation	Description
Start		Specifies the start point or beginning of an activity workflow.
Action		Specifies a step, task, or unit of work, and represents a discrete point in the process.
Decision		Represents a branch in the process flow where a control split occurs (Yes/No) or rejoins (converge).
Control Flow		Connects the nodes and represents the direction of flow of the process.
End		Specifies the end of the activity workflow.

ii. Theoretical Overview of the System’s Activity Workflows

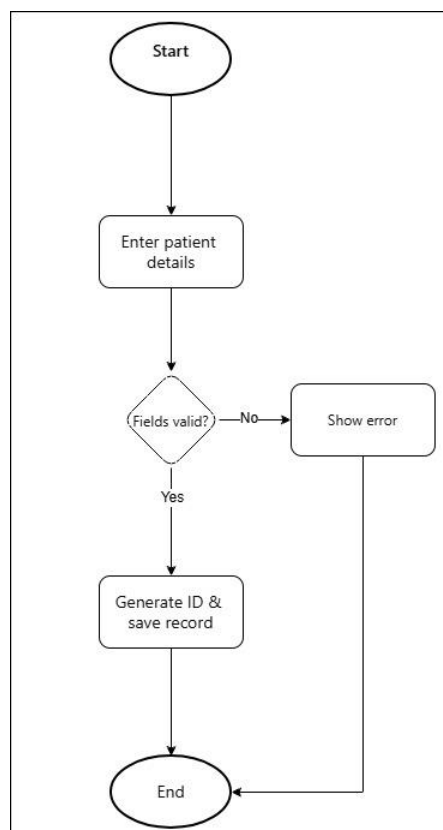
The system is provided with several core activity workflows which help to regulate and define the logic of users’ actions:

- **Authentication & Login Workflow**
 - ◆ **Credential Input:** Users enter their private system’s unique username and password in the login interface.
 - ◆ **Validity Inspection:** A decision node determines whether the login credentials exist in the database.
 - ◆ **Role-Based Routing:** If valid, the system routes users to the relevant dashboard based on their role (admin, doctor, or staff) and terminates the session (ends).



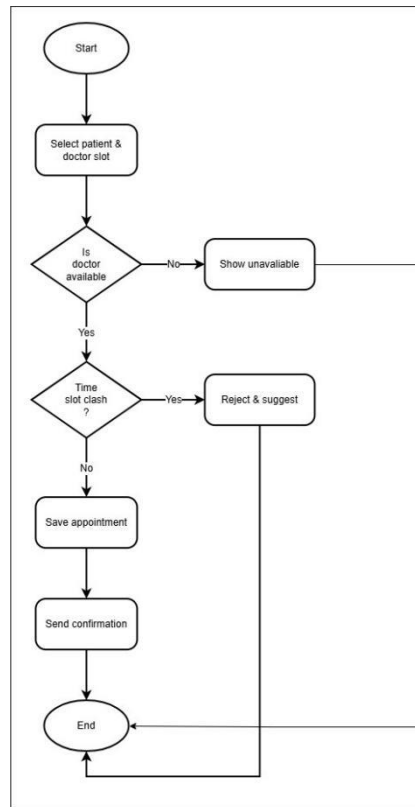
● 2. Patient Registration Workflow

- ◆ **Data Entry:** Front desk staff enter the incoming patient's personal and contact details in the system interface.
- ◆ **Field Validation:** The system validates the filled fields; if invalid, it triggers an error response.
- ◆ **Record Creation:** The staff saves the patient's data to generate the patient's ID and store the demographics in the database (store).



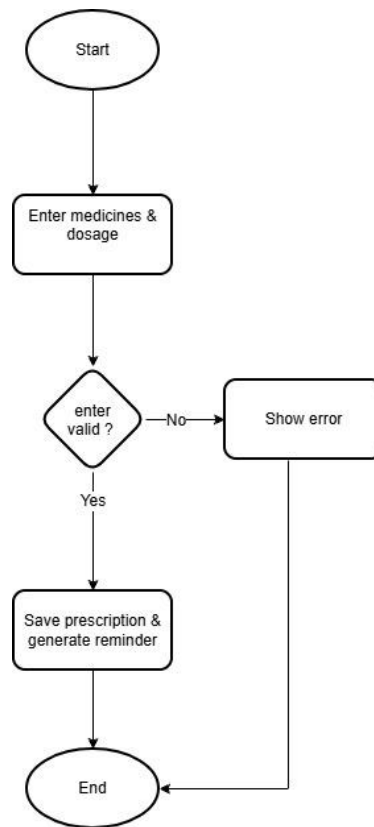
● Appointment Booking Workflow

- ◆ **Slot Selection:** The staff picks a patient's record and the preferred doctor and time slot for the service.
- ◆ **Availability & Clash Check:** The system confirms the availability of the selected doctor and checks for schedule clashes in the chosen time.
- ◆ **Confirmation or Rejection:** The system confirms the booking, sets reminders, and sends a confirmation message or rejects the request based on the results.



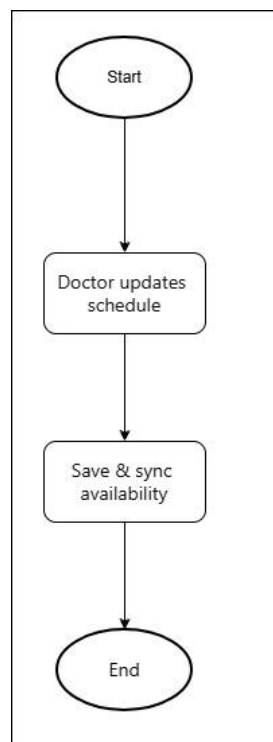
● Prescription Creation Workflow

- ◆ **Clinical Data Entry:** The doctor enters the drugs, doses, and notes of the prescriptions during the consultation.
- ◆ **Validation Check:** The system validates the prescription data entry; if invalid, it triggers an error response.
- ◆ **Persistence & Reminder Trigger:** The system stores the prescriptions in the database and triggers auto-reminder generation for the patient.



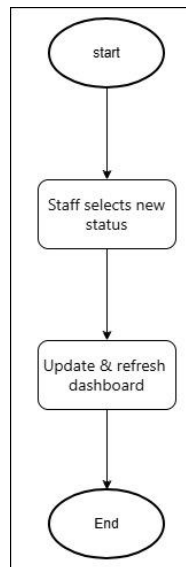
● Doctor Schedule & Availability

- ◆ **Workflow Schedule Modification:** Doctors update their schedules by setting the working and non-working hours.
- ◆ **Synchronization:** The system updates and stores the modified working hours in the database.
- ◆ **Operational Readiness:** The modified hours determine the doctors' availability for booking appointments.



● Appointment Status Update Workflow

- ◆ **Status Selection:** The staff selects the target appointment record and the new status option in the system interface.
- ◆ **Database Refresh:** The system updates the database with the new status and refreshes the dashboard interface.
- ◆ **Process Completion:** The process ends when the status has been updated on the staff and doctor's dashboards.



● Report Generation Workflow

- ◆ **Report Request:** Approved staff or doctors generate reports by requesting the required system or medical records.
- ◆ **Data Compilation:** The system compiles the records into a report document.
- ◆ **Export Execution:** The system exports the report document as a PDF file for printing and archiving

